CRM BLINDADORAS

Especificação Técnica Completa para o Claude Code

MVP	5 Módulos	Multi-tenant	SaaS
Fase 1 — 30 dias	Integrados e rastreados	Supabase RLS	Setup R$30k + R$2.5k/mês

Stack: Supabase · Antigravity · N8N · Evolution API · Stripe · DocuSign · Vercel (dev) · Hostinger (prod)
Versão: 1.0 | Data: 04/04/2026

0 Instruções Gerais para o Claude Code

Este documento é o briefing completo para construção do CRM Blindadoras. O Claude Code deve seguir exatamente a ordem das seções, sem pular etapas. Nenhum código deve ser executado manualmente pelo usuário.

Objetivo	Criar do zero um CRM SaaS multi-tenant para blindadoras, com projeto novo no Supabase, todas as tabelas, frontend em Antigravity e automações N8N.

Regras absolutas de desenvolvimento

- Criar projeto novo no Supabase via MCP (não reutilizar existente)
- Executar todos os SQL de criação de tabelas automaticamente
- Nunca pedir para o usuário rodar código no terminal
- Commitar e fazer deploy automático na Vercel ao final de cada fase
- Usar variáveis de ambiente para todas as credenciais sensíveis
- Seguir rigorosamente o design system definido na Seção 3

1 Stack Tecnológica & Ambiente

Camada	Tecnologia	Função
Backend / DB	Supabase	Auth, banco PostgreSQL, Storage, RLS, Realtime
Frontend	Antigravity	Interface completa responsiva
Automações	N8N (conta ativa)	Webhooks, notificações, integrações
WhatsApp	Evolution API	Notificações para clientes via WhatsApp
Pagamentos	Stripe	Cobrança SaaS mensal + setup
Assinatura Digital	DocuSign	Contratos e autorizações com validade jurídica
Deploy Dev	Vercel	Preview e testes — deploy automático via Git
Produção	Hostinger	Ambiente produtivo para o primeiro cliente

Variáveis de ambiente obrigatórias (.env)

```
NEXT_PUBLIC_SUPABASE_URL=NEXT_PUBLIC_SUPABASE_ANON_KEY=SUPABASE_SERVICE_ROLE_KEY=N8
N_WEBHOOK_BASE_URL=EVOLUTION_API_URL=EVOLUTION_API_KEY=STRIPE_SECRET_KEY=STRIPE_WEB
HOOK_SECRET=DOCUSIGN_INTEGRATION_KEY=DOCUSIGN_ACCOUNT_ID=NEXT_PUBLIC_APP_URL=
```

2 Supabase — Criação do Projeto & Banco de Dados

O Claude Code deve criar um projeto novo no Supabase usando o MCP do Supabase e executar todos os SQLs abaixo em sequência. Nenhum SQL deve ser executado manualmente.

Passo 2.1 — Criar projeto novo no Supabase

1. Usar a ferramenta MCP Supabase: create_project
2. Nome do projeto: crm-blindadoras
3. Região: sa-east-1 (São Paulo)
4. Aguardar confirmação de criação antes de prosseguir
5. Salvar as credenciais geradas nas variáveis de ambiente

Passo 2.2 — Extensões necessárias

```
-- Habilitar extensõesCREATE EXTENSION IF NOT EXISTS "uuid-ossp";CREATE EXTENSION
IF NOT EXISTS pgcrypto;
```

Passo 2.3 — Tabela: organizations (raiz multi-tenant)

```
CREATE TABLE organizations (  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  name
TEXT NOT NULL,  slug TEXT UNIQUE NOT NULL,  logo_url TEXT,  primary_color TEXT
DEFAULT '#22C55E',  domain TEXT,  plan TEXT DEFAULT 'active' CHECK (plan IN
('trial','active','suspended')),  setup_paid BOOLEAN DEFAULT FALSE,  monthly_value
NUMERIC(10,2) DEFAULT 2500.00,  stripe_customer_id TEXT,  stripe_subscription_id
TEXT,  created_at TIMESTAMPTZ DEFAULT NOW(),  updated_at TIMESTAMPTZ DEFAULT
NOW());
```

Passo 2.4 — Tabela: profiles (usuários)

```
CREATE TABLE profiles (  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE
CASCADE,  organization_id UUID REFERENCES organizations(id) ON DELETE CASCADE,
full_name TEXT NOT NULL,  email TEXT NOT NULL,  avatar_url TEXT,  role TEXT DEFAULT
'viewer' CHECK (role IN ('owner','admin','manager','technician','viewer')),
is_active BOOLEAN DEFAULT TRUE,  created_at TIMESTAMPTZ DEFAULT NOW(),  updated_at
TIMESTAMPTZ DEFAULT NOW());
```

Passo 2.5 — Tabela: leads

```
CREATE TABLE leads (  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
customer_name TEXT NOT NULL,  phone TEXT,  email TEXT,  plate TEXT,  vehicle_model
TEXT,  vehicle_year INT,  vehicle_color TEXT,  armor_type TEXT,  quoted_value
NUMERIC(10,2),  pipeline_stage TEXT DEFAULT 'new' CHECK (pipeline_stage IN
('new','prospecting','quoted','contracted')),  origin TEXT DEFAULT 'manual' CHECK
(origin IN ('manual','google_ads','meta','referral','website','whatsapp','other')),
notes TEXT,  assigned_to UUID REFERENCES profiles(id),  created_at TIMESTAMPTZ
DEFAULT NOW(),  updated_at TIMESTAMPTZ DEFAULT NOW());
```

Passo 2.6 — Tabela: projects (cards de blindagem)

```
CREATE TABLE projects (  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
lead_id UUID REFERENCES leads(id),  customer_name TEXT NOT NULL,  customer_phone
TEXT,  customer_email TEXT,  plate TEXT NOT NULL,  vehicle_model TEXT NOT NULL,
vehicle_year INT,  vehicle_color TEXT,  armor_type TEXT NOT NULL,  contract_value
NUMERIC(10,2) NOT NULL,  status TEXT DEFAULT 'in_progress' CHECK (status IN
('in_progress','completed','cancelled','on_hold')),  overall_progress NUMERIC(5,2)
DEFAULT 0,  odometer_in INT,  notes TEXT,  created_by UUID REFERENCES profiles(id),
created_at TIMESTAMPTZ DEFAULT NOW(),  updated_at TIMESTAMPTZ DEFAULT NOW());
```

Passo 2.7 — Tabela: production_stages

```
CREATE TABLE production_stages (  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
stage_number INT NOT NULL CHECK (stage_number BETWEEN 1 AND 12),  stage_name TEXT
NOT NULL,  percentage INT DEFAULT 0 CHECK (percentage BETWEEN 0 AND 100),  status
TEXT DEFAULT 'pending' CHECK (status IN
('pending','in_progress','completed','na')),  started_at TIMESTAMPTZ,  completed_at
TIMESTAMPTZ,  responsible_id UUID REFERENCES profiles(id),  notes TEXT,  created_at
TIMESTAMPTZ DEFAULT NOW(),  updated_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(project_id, stage_number));
```

Passo 2.8 — Tabela: stage_photos

```
CREATE TABLE stage_photos (  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
stage_id UUID NOT NULL REFERENCES production_stages(id) ON DELETE CASCADE,
organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
storage_path TEXT NOT NULL,  caption TEXT,  photo_type TEXT DEFAULT 'progress'
CHECK (photo_type IN ('before','during','after','damage','progress')),  taken_at
TIMESTAMPTZ DEFAULT NOW(),  uploaded_by UUID REFERENCES profiles(id));
```

Passo 2.9 — Tabela: materials (catálogo de estoque)

```
CREATE TABLE materials (  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,  name
TEXT NOT NULL,  category TEXT DEFAULT 'other' CHECK (category IN
('glass','steel','aramid','hardware','paint','finishing','other')),  unit TEXT
DEFAULT 'un' CHECK (unit IN ('un','kg','m','m2','L','cx')),  unit_cost
NUMERIC(10,2) NOT NULL DEFAULT 0,  stock_quantity NUMERIC(10,3) DEFAULT 0,
```

```
min_stock NUMERIC(10,3) DEFAULT 0,  supplier TEXT,  sku TEXT,  is_active BOOLEAN
DEFAULT TRUE,  created_at TIMESTAMPTZ DEFAULT NOW(),  updated_at TIMESTAMPTZ
DEFAULT NOW());
```

Passo 2.10 — Tabela: project_materials (consumo por projeto)

```
CREATE TABLE project_materials (  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,  material_id
UUID NOT NULL REFERENCES materials(id),  organization_id UUID NOT NULL REFERENCES
organizations(id) ON DELETE CASCADE,  quantity NUMERIC(10,3) NOT NULL,  unit_cost
NUMERIC(10,2) NOT NULL,  total_cost NUMERIC(10,2) GENERATED ALWAYS AS (quantity *
unit_cost) STORED,  added_by UUID REFERENCES profiles(id),  added_at TIMESTAMPTZ
DEFAULT NOW());
```

Passo 2.11 — Tabela: documents

```
CREATE TABLE documents (  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
doc_type TEXT NOT NULL CHECK (doc_type IN
('authorization','declaration','contract','proposal','other')),  status TEXT
DEFAULT 'awaiting_docs' CHECK (status IN
('awaiting_docs','awaiting_approval','approved','rejected')),  docusign_envelope_id
TEXT,  storage_path TEXT,  notes TEXT,  approved_by UUID REFERENCES profiles(id),
approved_at TIMESTAMPTZ,  created_at TIMESTAMPTZ DEFAULT NOW(),  updated_at
TIMESTAMPTZ DEFAULT NOW());
```

Passo 2.12 — Tabela: financials

```
CREATE TABLE financials (  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
project_id UUID REFERENCES projects(id),  type TEXT NOT NULL CHECK (type IN
('receivable','payable')),  description TEXT NOT NULL,  amount NUMERIC(10,2) NOT
NULL,  payment_method TEXT CHECK (payment_method IN
('pix','boleto','cash','card','installment','transfer')),  status TEXT DEFAULT
'pending' CHECK (status IN ('pending','paid','partial','overdue','cancelled')),
due_date DATE,  paid_at TIMESTAMPTZ,  supplier_id UUID,  notes TEXT,  created_at
TIMESTAMPTZ DEFAULT NOW(),  updated_at TIMESTAMPTZ DEFAULT NOW());
```

Passo 2.13 — Tabela: maintenance_orders

```
CREATE TABLE maintenance_orders (  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
project_id UUID REFERENCES projects(id),  customer_name TEXT NOT NULL,  plate TEXT
NOT NULL,  vehicle_model TEXT,  type TEXT DEFAULT 'revision' CHECK (type IN
('revision','repair','warranty','inspection')),  status TEXT DEFAULT 'open' CHECK
(status IN ('open','in_progress','completed','cancelled')),  diagnosis TEXT,  cost
NUMERIC(10,2),  scheduled_at TIMESTAMPTZ,  completed_at TIMESTAMPTZ,
responsible_id UUID REFERENCES profiles(id),  created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW());
```

Passo 2.14 — Função: calcular progresso geral do projeto

```
CREATE OR REPLACE FUNCTION update_project_progress()RETURNS TRIGGER AS $$BEGIN
UPDATE projects SET    overall_progress = (      SELECT COALESCE(AVG(percentage),
0)      FROM production_stages      WHERE project_id = NEW.project_id    ),
updated_at = NOW()  WHERE id = NEW.project_id;  RETURN NEW;END;$$ LANGUAGE
plpgsql;CREATE TRIGGER trg_update_project_progressAFTER INSERT OR UPDATE ON
production_stagesFOR EACH ROW EXECUTE FUNCTION update_project_progress();
```

Passo 2.15 — Função: calcular custo de materiais do projeto

```
CREATE OR REPLACE FUNCTION get_project_material_cost(p_id UUID)RETURNS NUMERIC AS
$$  SELECT COALESCE(SUM(total_cost), 0)  FROM project_materials  WHERE project_id =
p_id;$$ LANGUAGE sql STABLE;
```

Passo 2.16 — Seed: 12 etapas padrão (função automática)

```
CREATE OR REPLACE FUNCTION create_default_stages()RETURNS TRIGGER AS $$DECLARE
stage_names TEXT[] := ARRAY[    'Aguardando Coleta', 'Checklist Inicial',
'Preparação',    'Desmontagem', 'Desenvolvimento', 'Aplicação Parte Opaca',
'Aguardando Vidros', 'Aplicação Parte Transparente',    'Montagem e Finalização',
'Testes Finais',    'Preparação Estética', 'Entrega'  ];  i INT;BEGIN  FOR i IN
1..12 LOOP    INSERT INTO production_stages (project_id, organization_id,
stage_number, stage_name)    VALUES (NEW.id, NEW.organization_id, i,
stage_names[i]);  END LOOP;  RETURN NEW;END;$$ LANGUAGE plpgsql;CREATE TRIGGER
trg_create_default_stagesAFTER INSERT ON projectsFOR EACH ROW EXECUTE FUNCTION
create_default_stages();
```

Passo 2.17 — RLS: Row Level Security (multi-tenant)

```
-- Habilitar RLS em todas as tabelasALTER TABLE organizations ENABLE ROW LEVEL
SECURITY;ALTER TABLE profiles ENABLE ROW LEVEL SECURITY;ALTER TABLE leads ENABLE
ROW LEVEL SECURITY;ALTER TABLE projects ENABLE ROW LEVEL SECURITY;ALTER TABLE
production_stages ENABLE ROW LEVEL SECURITY;ALTER TABLE stage_photos ENABLE ROW
LEVEL SECURITY;ALTER TABLE materials ENABLE ROW LEVEL SECURITY;ALTER TABLE
project_materials ENABLE ROW LEVEL SECURITY;ALTER TABLE documents ENABLE ROW LEVEL
SECURITY;ALTER TABLE financials ENABLE ROW LEVEL SECURITY;ALTER TABLE
maintenance_orders ENABLE ROW LEVEL SECURITY;-- Política base: usuário vê apenas
dados da sua organizaçãoCREATE POLICY "org_isolation" ON leads USING (
organization_id = (SELECT organization_id FROM profiles WHERE id = auth.uid()));--
Repetir política para todas as tabelas substituindo 'leads' por cada tabela
```

Passo 2.18 — Storage Buckets

```
-- Criar buckets no Supabase Storage-- Executar via MCP Supabase storage API:--
Bucket: stage-photos (público: false, max size: 10MB, tipos: image/*)-- Bucket:
documents (público: false, max size: 25MB, tipos: application/pdf)-- Bucket:
org-assets (público: true, max size: 2MB, tipos: image/*)
```

3 Design System — Identidade Visual

O design deve ser fiel às referências visuais fornecidas: sidebar escura, área de conteúdo branca, detalhes em verde, bordas arredondadas, tipografia limpa.

Paleta de cores

Token	Hex	Uso
--color-primary	#22C55E	Botões primários, badges ativos, destaques
--color-primary-dark	#16A34A	Hover dos botões verdes
--color-sidebar-bg	#111827	Fundo da sidebar (dark)
--color-sidebar-text	#D1D5DB	Texto dos itens de menu
--color-sidebar-active	#22C55E	Item de menu ativo
--color-bg-main	#FFFFFF	Background principal da área de conteúdo
--color-bg-card	#FFFFFF	Cards e painéis
--color-bg-secondary	#F9FAFB	Linhas alternadas de tabela, inputs
--color-border	#E5E7EB	Bordas de cards e tabelas
--color-text-primary	#111827	Títulos e textos principais
--color-text-secondary	#6B7280	Subtítulos e labels
--color-success	#22C55E	Status concluído
--color-warning	#F59E0B	Status em andamento
--color-danger	#EF4444	Status vencido / erro
--color-info	#3B82F6	Informativo

Tipografia

- Fonte principal: Inter (Google Fonts)
- Fallback: system-ui, -apple-system, sans-serif
- Tamanhos: 12px (legenda), 14px (body small), 16px (body), 20px (subtitle), 24px (heading), 32px (KPI)
- Peso: 400 (regular), 500 (medium), 600 (semibold), 700 (bold)

Bordas e sombras

- border-radius padrão: 12px (cards), 8px (buttons, inputs, badges), 6px (pequenos)
- box-shadow cards: 0 1px 3px rgba(0,0,0,0.08), 0 1px 2px rgba(0,0,0,0.04)

- box-shadow hover: 0 4px 12px rgba(0,0,0,0.10)
- Transição padrão: all 0.15s ease

Layout base

- Sidebar fixa: 240px largura, fundo #111827, altura 100vh
- Área de conteúdo: calc(100vw - 240px), padding 24px
- Header de página: altura 64px, border-bottom 1px solid #E5E7EB
- Grid de KPI cards: 4 colunas (desktop), 2 (tablet), 1 (mobile)
- Gap padrão entre elementos: 16px

4 Estrutura de Páginas — MVP Fase 1

4.1 — Componentes globais (criar primeiro)

- Sidebar navigation component com logo, menu items, user profile
- TopBar component com título da página, botões de ação, avatar
- KPI Card component (valor, label, variação percentual, ícone)
- Status Badge component (cores por status: verde/amarelo/laranja/vermelho/cinza)
- Modal base component (overlay, header, body, footer com actions)
- Loading skeleton components para todos os cards
- Toast / notification component
- Empty state component

4.2 — Página: Login / Auth

- Layout: tela dividida — lado esquerdo verde com logo, lado direito formulário branco
- Campos: email, senha, checkbox lembrar-me
- Botão: Entrar (verde, width 100%)
- Link: Esqueci minha senha
- Integração: Supabase Auth (signInWithPassword)
- Redirect automático para /dashboard após login
- Proteção de rotas: HOC ou middleware que verifica sessão

4.3 — Página: Dashboard (/dashboard)

- Greeting: 'Olá, [nome]! Aqui está o resumo do seu dia.'
- Row de 4 KPI Cards: Leads do mês, Taxa de conversão, Receita do mês, Projetos ativos
- Gráfico de barras: Receita mensal dos últimos 12 meses (Chart.js ou Recharts)

- Tabela: Projetos recentes com status e progresso
- Mini-calendário: próximas entregas e manutenções
- Dados em tempo real via Supabase Realtime

4.4 — Página: Pipeline de Vendas (/pipeline)

- Kanban drag-and-drop com 4 colunas: NOVO (vermelho), PROSPECÇÃO (amarelo), ORÇADO (laranja), CONTRATADO (verde)
- Cada card do kanban exibe: nome cliente, placa, modelo, tipo blindagem, valor orçado
- Botão flutuante: + Adicionar Lead
- Modal: Novo Lead — campos: nome, telefone, email, placa, modelo, ano, cor, tipo blindagem, valor orçado, origem
- Ao arrastar para CONTRATADO: criar automaticamente um Project na tabela projects
- Contador de cards por coluna no header de cada coluna
- Biblioteca de drag-and-drop: @dnd-kit/core (leve, acessível)

4.5 — Página: Projetos / Cards de Blindagem (/projects)

- Grid de cards com busca e filtros (status, técnico responsável, período)
- Cada card exibe: placa, cliente, modelo, progresso geral (barra verde), etapa atual
- Clicar no card abre a página de detalhe do projeto

4.6 — Página: Detalhe do Projeto (/projects/[id])

Esta é a página mais complexa do sistema — o Card de Projeto expandido.

- SEÇÃO 1: Dados do cliente e veículo (edição inline)
- SEÇÃO 2: 12 barras de progresso coloridas (cinza / amarelo / verde) com percentual
- Barra geral de progresso no topo (média das 12 etapas)
- Botão 'Editar Etapas': abre modal com 12 sliders (0-100%) + responsável + data + notas
- SEÇÃO 3: Financeiro — valor contratado, custo materiais, lucro bruto, % lucro
- SEÇÃO 4: Materiais utilizados — lista com expandir/recolher, total calculado
- Botão 'Adicionar Materiais': modal com select de material do catálogo + quantidade
- SEÇÃO 5: Documentações — Autorização e Declaração com troca de status visual
- SEÇÃO 6: Botões de ação — Editar Etapas (azul), Adicionar Materiais (verde), Documentações (roxo), Finalizar Projeto (verde escuro)
- Timeline lateral: histórico de alterações do projeto

4.7 — Página: Documentações (/documents)

- Lista de todos os documentos de todos os projetos
- Filtros: tipo (autorização/declaração/contrato), status, projeto

- Ações: Visualizar, Aprovar, Rejeitar, Download PDF
- Integração DocuSign: botão 'Solicitar Assinatura' chama API DocuSign

5 Automações N8N — Configuração

Criar os seguintes workflows no N8N. O Claude Code deve gerar os JSON de importação de cada workflow.

Workflow	Gatilho	Ação
Novo Lead Criado	Webhook Supabase (INSERT leads)	Notificar vendedor responsável via Evolution API
Etapa Concluída	Webhook Supabase (UPDATE production_stages status=completed)	WhatsApp para cliente com progresso atual
Documento Aprovado	Webhook Supabase (UPDATE documents status=approved)	E-mail + WhatsApp para admin e cliente
Estoque Mínimo	Cron diário 08:00h — query materials stock	WhatsApp para admin com lista de itens críticos
Contrato Fechado	Webhook (UPDATE leads stage=contracted)	Criar projeto automaticamente via Supabase API
Pagamento Vencido	Cron diário 09:00h — query financials vencidos	Alerta para admin + E-mail para cliente

Configuração Evolution API no N8N

- Criar credencial HTTP no N8N com Base URL da Evolution API e API Key
- Instância WhatsApp: criar instância 'crm-blindadoras' e conectar QR Code
- Template de mensagem padrão: '[NomeEmpresa] — Seu veículo [Placa] avançou para a etapa [X]. Progresso: [Y]%'

6 Deploy — Vercel (Dev) & Hostinger (Prod)

Passo 6.1 — Deploy Vercel (desenvolvimento)

6. Criar repositório Git e fazer push inicial do projeto
7. Conectar repositório à Vercel via CLI: vercel --prod
8. Configurar todas as variáveis de ambiente na Vercel dashboard

9. Ativar Preview Deployments para cada PR/branch
10. Configurar domínio preview: crm-dev.vercel.app

Passo 6.2 — Deploy Hostinger (produção — após validação)

11. Apontar domínio do cliente para Hostinger via DNS
12. Configurar Node.js hosting ou usar build estático (Next.js export)
13. Copiar variáveis de ambiente de produção
14. Configurar SSL automático (Let's Encrypt via Hostinger)
15. Testar todas as funcionalidades no ambiente produtivo

7 Ordem Exata de Execução — MVP 30 Dias

Dia	Entrega	Detalhes	Prioridade
1-2	Setup Supabase	Criar projeto, executar todos os SQLs, configurar RLS, storage buckets	CRÍTICO
3-4	Design System	Tokens CSS, componentes base, sidebar, topbar, tipografia	CRÍTICO
5-6	Auth / Login	Tela de login, proteção de rotas, multi-tenant por organização	CRÍTICO
7-9	Dashboard	KPI cards, gráfico de barras, tabela de projetos recentes	ALTO
10-13	Pipeline Kanban	Drag-and-drop 4 colunas, modal novo lead, conversão para projeto	ALTO
14-19	Card de Projeto	12 etapas, modal edição, materiais, financeiro, documentações	ALTO
20-22	Documentações	Página de docs, status visual, integração DocuSign básica	MÉDIO
23-25	N8N Automações	5 workflows principais, Evolution API WhatsApp	MÉDIO
26-27	Responsivo	Ajustes mobile, testar pipeline no tablet, testar formulários	MÉDIO
28-29	Testes & Bugs	Teste end-to-end com dados reais do cliente, correção de bugs	ALTO
30	Deploy Vercel	Deploy final, variáveis de produção, entrega ao cliente	CRÍTICO

8 Checklist Final de Entrega

Banco de dados

- Projeto Supabase criado e credenciais salvas
- Todas as 11 tabelas criadas e testadas
- RLS ativo em todas as tabelas
- Triggers de progresso e etapas funcionando
- Storage buckets criados
- Dados seed: 1 organização de teste + 1 usuário admin

Frontend

- Design system implementado (cores, tipografia, bordas)
- Dark sidebar + white content area
- Login funcional com Supabase Auth
- Dashboard com KPIs reais do banco
- Pipeline kanban com drag-and-drop
- Card de projeto com 12 etapas
- Modal de materiais com cálculo de lucro em tempo real
- Documentações com troca de status
- Totalmente responsivo (mobile + tablet + desktop)

Automações & integrações

- N8N: 5 workflows ativos e testados
- Evolution API: instância WhatsApp conectada
- DocuSign: integração básica de solicitação de assinatura
- Stripe: configurado para cobrança futura (fase 2)

Deploy

- Repositório Git organizado com .gitignore correto
- Variáveis de ambiente configuradas na Vercel
- Build sem erros ou warnings críticos
- URL de preview compartilhada com o cliente